

Cassandra vs SQL: El cambio de mentalidad (Query-First)

Departamento de Informática

22 de abril de 2026

1 Cassandra vs SQL: El cambio de mentalidad (Query-First)

1.1 Contexto de Negocio: Registro de Búsquedas (GloboTour)

En la plataforma **GloboTour**, miles de usuarios realizan búsquedas de destinos cada segundo. Guardar esto en una base de datos relacional (como MySQL) bloquearía las tablas debido al alto volumen de escritura.

Usamos **Cassandra** para este módulo de «Logs de Búsqueda» porque:

1. **Escalabilidad:** Podemos añadir servidores fácilmente si el tráfico sube.
2. **Escritura ultra-rápida:** Cassandra está diseñada para insertar millones de registros sin despeinarse.
3. **Consulta específica:** El equipo de Marketing necesita ver rápidamente el historial de un usuario para ofrecerle ofertas personalizadas.

1.1.1 El Modelo de Datos Creado

La tabla `user_search_logs` está diseñada para responder a: «¿Qué ha buscado el usuario X recientemente?»

- **Partition Key (`user_id`):** Agrupa todas las búsquedas de un usuario en el mismo nodo.
 - **Clustering Key (`search_time`):** Ordena esas búsquedas automáticamente de la más reciente a la más antigua.
-

1.2 1. La Gran Diferencia: ¿Dónde están mis JOINS?

En SQL normalizamos para evitar duplicar datos. En Cassandra **la duplicación es tu amiga** para ganar velocidad.

1.2.1 El problema del JOIN

- **SQL:** Tienes una tabla `usuarios` y una tabla `busquedas`. Para saber qué búsquedas hizo «Víctor», haces un JOIN.
- **Cassandra:** **NO existen los JOINS.** Cruzar datos entre nodos en un cluster de 100 servidores sería lentísimo.

1.2.2 La solución: Desnormalización (Duplicar info)

Si quieres mostrar el nombre del usuario junto a su búsqueda, **guardas el nombre del usuario dentro de la tabla de búsquedas**, aunque se repita 1000 veces.

1.3 Comparativa de Modelado (Caso GloboTour)

Imagina que queremos saber: 1. «Todas las búsquedas del usuario X». 2. «Todas las búsquedas que se han hecho al destino “París”».

1.3.1 En SQL (Relacional)

Tendrías una sola tabla y harías:

```
SELECT * FROM busquedas WHERE usuario_id = 'user1'; -- Indice en usuario_id
SELECT * FROM busquedas WHERE destino = 'Paris'; -- Indice en destino
```

1.3.2 En Cassandra (Columnar)

Necesitas **DOS TABLAS** para los mismos datos:

Tabla A: Optimizada para buscar por USUARIO

```
CREATE TABLE busquedas_por_usuario (
  usuario_id text,
  fecha timestamp,
  destino text,
  nombre_usuario text, -- DATO DUPLICADO (Desnormalizado)
  PRIMARY KEY (usuario_id, fecha)
) WITH CLUSTERING ORDER BY (fecha DESC);
```

Tabla B: Optimizada para buscar por DESTINO

```
CREATE TABLE busquedas_por_destino (
  destino text,
  fecha timestamp,
  usuario_id text,
  PRIMARY KEY (destino, fecha)
);
```

1.4 ¿Qué pasa si necesito un JOIN «sí o sí»?

Si te encuentras en una situación donde necesitas datos de dos tablas, tienes tres caminos en Cassandra:

1. **Duplicar la información (Recomendado):** Añade las columnas que necesites de la Tabla B en la Tabla A en el momento de la inserción. (Espacio en disco barato vs Tiempo de CPU caro).
 2. **JOIN en la Aplicación:** Tu programa (Python/Java) hace una consulta a la Tabla A, obtiene un ID, y luego hace otra consulta a la Tabla B. (Solo para datos que cambian poco).
 3. **Tablas Complementarias:** Crear una tabla específica para esa «vista» concreta que necesitas.
-

1.5 ¿Hay Foreign Keys (FK) en Cassandra?

NO. No existe la integridad referencial.

- **¿Por qué?:** En un sistema distribuido, comprobar si un ID existe en otro nodo antes de insertar un dato sería extremadamente lento.
 - **Consecuencia:** Puedes insertar una búsqueda para un `usuario_id` que no existe.
 - **Responsabilidad:** La integridad de los datos recae ahora en el **programador** (la lógica de la aplicación), no en la base de datos.
-

1.6 Ejemplos de consultas que NO puedes hacer (y por qué)

1.6.1 Error 1: Filtrar por algo que no es Clave de Partición

```
-- ESTO DARÁ ERROR
SELECT * FROM busquedas_por_usuario WHERE destino = 'Madrid';
```

- **Por qué:** Cassandra no sabe en qué nodo está «Madrid». Tendría que escanear todo el cluster.
- **Solución:** Crear la tabla `busquedas_por_destino`.

1.6.2 Error 2: Usar el Operador «OR»

```
-- ESTO NO EXISTE EN CASSANDRA
SELECT * FROM busquedas_por_usuario WHERE usuario_id = 'u1' OR usuario_id = 'u2';
```

- **Por qué:** Cada usuario puede estar en un servidor distinto. Las consultas deben ser precisas.
- **Solución:** Usar el operador `IN` o hacer dos consultas separadas.

1.6.3 Error 3: El «SELECT COUNT(*)»

```
-- ESTO ES MUY LENTO
SELECT COUNT(*) FROM busquedas_por_usuario;
```

- **Por qué:** Cassandra tiene que recorrer todos los nodos.
- **Alternativa (Contadores):** Crear una tabla específica de tipo counter.

```
CREATE TABLE estadisticas_destino (
    destino text PRIMARY KEY,
    total counter
);

-- Actualizar el contador (no se usa INSERT, sino UPDATE)
UPDATE estadisticas_destino SET total = total + 1 WHERE destino = 'Paris';
```

1.7 Consultas que SÍ puedes hacer (y cómo pensar en ellas)

Paso previo obligatorio

Antes de lanzar cualquier consulta en la consola (`cqlsh`), debes seleccionar el Keyspace de la empresa:

```
USE globotour;
```

Si no lo haces, Cassandra te devolverá el error: «*No keyspace has been specified*».

Para que una consulta funcione en Cassandra, siempre debemos empezar por la **Partition Key**. En nuestra tabla `busquedas_por_usuario`, la PK es `user_id`.

1.7.1 Buscar todas las búsquedas de un usuario específico

Esta es la consulta para la que se diseñó la tabla. Es instantánea.

```
SELECT * FROM busquedas_por_usuario
WHERE user_id = 'user_1';
```

1.7.2 Buscar en un rango de tiempo (Clustering Column)

Como `search_time` es una clustering column, podemos hacer filtros de rango **siempre** que hayamos fijado el usuario primero.

```
-- Búsquedas de user_1 realizadas después del 15 de abril
SELECT * FROM busquedas_por_usuario
WHERE user_id = 'user_1'
AND search_time > '2026-04-15 00:00:00';
```

1.7.3 Limitar resultados (Top N)

Gracias al orden físico (`CLUSTERING ORDER BY search_time DESC`), obtener las últimas búsquedas es muy eficiente.

```
-- Las 3 últimas búsquedas de user_7
SELECT * FROM busquedas_por_usuario
WHERE user_id = 'user_7'
LIMIT 3;
```

1.7.4 El truco del ALLOW FILTERING (Uso con precaución)

Si realmente necesitas buscar por algo que no es clave (como el destino), Cassandra te permite forzarlo, pero te advierte que será lento en un entorno real.

```
-- Buscar búsquedas a Madrid de CUALQUIER usuario
SELECT * FROM busquedas_por_usuario
WHERE destino = 'Madrid'
ALLOW FILTERING;
```

1.8 Visualización Profesional con CloudBeaver

Para ver cómo se organizan los datos realmente en el clúster:

1. Abre `http://localhost:8978`.
2. Selecciona la conexión **GloboTour**.
3. Explora la tabla `busquedas_por_usuario`. Verás que los datos están agrupados por partición, lo cual es la clave del rendimiento de Cassandra.

1.9 Desafíos de Aprendizaje (Para practicar en clase)

1. **La Consulta Prohibida:** Intenta buscar una búsqueda filtrando solo por `destino` en la tabla `busquedas_por_usuario`. ¿Qué error te da Cassandra? ¿Cómo lo solucionarías profesionalmente?
2. **Orden Físico:** Inserta tres búsquedas para el mismo usuario pero con fechas muy distintas. Haz un `SELECT`. ¿En qué orden aparecen? ¿Por qué?
3. **Integridad entre Tablas:** Si duplicamos los datos en dos tablas (`busquedas_por_usuario` y `busquedas_por_destino`), ¿cómo asegura Cassandra que si borro un registro en una se borre en la otra?
4. **Ampliación Avanzada:** Una vez dominados estos conceptos básicos, dirígete al **02-Laboratorio_QueryFirst.tex** para aprender patrones industriales como Series Temporales y TTL.

1.10 Solucionario de Desafíos

Solución Desafío 1: El Filtro Imposible

Si ejecutas `SELECT * FROM busquedas_por_usuario WHERE destino = 'Madrid';`, obtendrás un error indicando que Cassandra no puede realizar el escaneo (*Cannot execute this query as it might involve data filtering and thus may have unpredictable performance*).

Solución Profesional: Crear una tabla específica para esa pregunta:

```
CREATE TABLE busquedas_por_destino (  
    destino text,  
    search_time timestamp,  
    user_id text,  
    PRIMARY KEY (destino, search_time)  
);
```

Solución Desafío 2: El Orden de los Datos

Al hacer el `SELECT`, las búsquedas aparecerán ordenadas de **más reciente a más antigua**.

Por qué: En la definición de la tabla incluimos la cláusula `WITH CLUSTERING ORDER BY (search_time DESC)`. Cassandra guarda los datos físicamente en ese orden dentro del disco de cada nodo, lo que hace que recuperar el historial reciente sea extremadamente rápido.

Solución Desafío 3: Coherencia y ACID

Respuesta Corta: No lo hace. Cassandra no tiene claves ajenas (FK) ni transacciones ACID entre tablas.

Explicación: Si borras un dato en una tabla, la otra seguirá teniéndolo a menos que tu aplicación haga los dos borrados. Para asegurar que ambos cambios ocurren «a la vez» o no ocurre ninguno, Cassandra ofrece los **BATCHES** (lotes), aunque su uso excesivo puede bajar el rendimiento. En el mundo NoSQL, se suele aceptar la «Consistencia Eventual». ¿Recuerdas lo que es?